# Kit móvil de atención primaria en salud

## Guía de despliegue

**Proyecto final - Plataformas I - 2026-2** · Organización `kitsalud-movil-plats1`

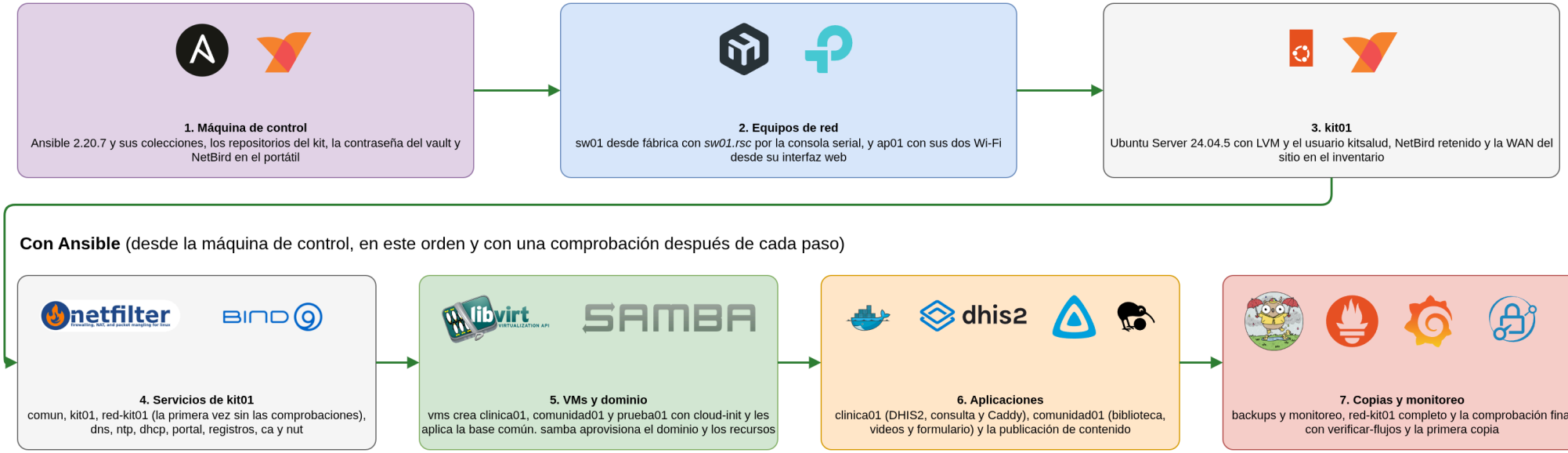
Esta guía lleva el kit desde equipos recién sacados de la caja hasta todos sus servicios funcionando, y explica cómo reconstruir una máquina virtual cuando se pierde. Está escrita para un administrador que conoce Linux y no participó en la instalación. Casi todo lo hace Ansible desde una máquina de control, así que la guía dice qué ejecutar, en qué orden, por qué ese orden y cómo comprobar cada paso antes de seguir. Cada parte trae los paquetes que instala, los servicios que quedan activos y los archivos de configuración que escribe.

| Parte | Para qué |
|---|---|
| **Preparación** | Dejar lista la máquina de control con Ansible, los repositorios y los secretos |
| **Equipos de red** | Conectar los equipos y configurar el switch y el punto de acceso |
| **kit01** | Instalar Ubuntu en el mini PC y desplegar la red, el firewall y los servicios del host |
| **Máquinas virtuales** | Crear clinica01 y comunidad01 y desplegar sus aplicaciones, la publicación, las copias y el monitoreo |
| **Orden de inicio** | Qué arranca primero cuando se enciende el kit y de qué depende cada servicio |
| **Comprobación final** | Confirmar que el kit quedó completo y resolver los errores más comunes del despliegue |
| **Reconstruir una máquina virtual** | Volver a crear una VM desde cero, con el resultado de haberlo hecho en el kit |

## Las fases del despliegue

El despliegue tiene siete fases. Las tres primeras se hacen una vez a mano, en la consola o el navegador de cada equipo, y las otras cuatro las hace Ansible desde la máquina de control.

**A mano** (una vez, con consola o navegador)




**Con Ansible** (desde la máquina de control, en este orden y con una comprobación después de cada paso)




Cada playbook termina en un PLAY RECAP sin *failed* ni *unreachable*. Si uno falla, se corrige la causa y se vuelve a ejecutar el mismo, porque todos son idempotentes. La red y el firewall de kit01 se aplican con una restauración programada que vuelve a la configuración anterior si se pierde el acceso.

## El kit en pocas palabras

| Equipo | Qué es | Dirección |
|---|---|---|
| **kit01** | Mini PC Beelink EQi12 que hace de router, firewall, DNS, DHCP, hora, portal cautivo, logs, monitoreo y copias, y donde corren las dos máquinas virtuales | `10.20.10.1` desde la red Interna, `100.90.225.113` por NetBird |
| **clinica01** | Máquina virtual con la historia clínica (DHIS2) y su base de datos, el dominio de usuarios, los archivos compartidos y la consulta de prerregistros | `10.20.20.11` y `fd5a:fc7e:d716:20::11` |
| **comunidad01** | Máquina virtual con la biblioteca, los videos y el formulario de prerregistro | `10.20.20.12` y `fd5a:fc7e:d716:20::12` |
| **sw01** | Switch MikroTik CCR2004 que separa la red Interna y la Comunidad | `10.20.10.2` |
| **ap01** | Punto de acceso TP-Link TL-WA801ND con un Wi-Fi para cada red | `10.20.10.3` |

# Preparación

Todo el despliegue se ejecuta desde una máquina de control, que es el portátil del administrador. Esta parte la deja lista con Ansible, los repositorios del kit y los secretos, y comprueba que llega a los equipos. Se hace una vez por máquina.

## Lo que se necesita

| Qué | Para qué |
|---|---|
| **Portátil con Linux (Ubuntu 24.04 o similar), Internet y sudo** | Máquina de control. En macOS los paquetes se instalan con Homebrew |
| **Una cuenta en la red de NetBird del grupo y su clave de registro** | Llegar a kit01 en remoto, durante el despliegue y después |
| **La contraseña del vault, que el grupo comparte por un canal privado** | Abrir los secretos del kit (contraseñas de los usuarios, de las bases y de las copias) |
| **Memoria USB de 4 GB o más** | Instalador de Ubuntu Server 24.04.5 para kit01 |
| **Cable de consola RJ45 a USB** | Configurar el switch desde cero |
| **Disco USB para las copias** | Repositorio de restic, conectado a kit01 |

## 1. Paquetes de la máquina de control

Ansible se instala con `pipx` en la misma versión que se usó para desplegar el kit, y las colecciones también con versión fija.

```
sudo apt-get update
sudo apt-get install -y git sshpass openssh-client pipx
pipx install ansible-core==2.20.7
export PATH="$HOME/.local/bin:$PATH"          # o abrir una terminal nueva después de "pipx
        ensurepath"
ansible-galaxy collection install ansible.posix:==2.2.0 community.general:==12.6.2
        community.crypto:==3.2.2
```

| Paquete | Versión | Para qué |
|---|---|---|
| **`ansible-core`** | 2.20.7 | Ejecutar los playbooks |
| **`ansible.posix`, `community.general`, `community.crypto`** | 2.2.0, 12.6.2, 3.2.2 | Montajes, LVM, ufw, zona horaria y la CA del kit |

| Paquete | Versión | Para qué |
|---|---|---|
| **`sshpass`** | La de la distribución | Entrar con contraseña a kit01 y, saltando por kit01, a las máquinas virtuales |
| **`git`** | La de la distribución | Clonar los repositorios |

### Cómo comprobarlo

```
ansible --version | head -1                       # ansible [core 2.20.7]
ansible-galaxy collection list | grep -E 'ansible.posix|community.(general|crypto)'
```

## 2. Repositorios

Los roles de Ansible viven en cuatro repositorios distintos y se encuentran entre sí por su posición, así que se clonan uno al lado del otro dentro del espacio de trabajo. El script `clonar-repos.sh` lo hace y, si ya existen, los actualiza.

```
git clone https://github.com/kitsalud-movil-plats1/workspace.git kitsalud
kitsalud/scripts/clonar-repos.sh
```

| Carpeta | Qué tiene |
|---|---|
| **`kitsalud/network`** | Red y firewall de kit01, DHCP, anuncios IPv6, portal cautivo, configuración del switch y revisión del punto de acceso |
| **`kitsalud/platform`** | Ansible (inventario, secretos y playbooks) y los roles de la base de kit01, las máquinas virtuales, DNS, hora, dominio, CA, publicación, copias y UPS |
| **`kitsalud/apps`** | Roles de las aplicaciones de clinica01 y comunidad01 (Docker, DHIS2, Caddy, Kiwix, Jellyfin, formularios y consulta) |
| **`kitsalud/observability`** | Roles de Prometheus, los exporters, Grafana y el receptor de logs |

Ansible se ejecuta siempre desde `kitsalud/platform/ansible`, porque ahí está `ansible.cfg` con el inventario, las rutas de los roles y la ubicación de la contraseña del vault.

## 3. Secretos

Las contraseñas del kit están cifradas con `ansible-vault` en `inventarios/kit/group_vars/all/vault.yml`, que sí está en el repositorio. Lo que no está es la contraseña que lo abre. Se guarda en la máquina de control con permisos `600`.

```
install -d -m 700 ~/.config/kitsalud
(umask 077; read -rsp 'Contraseña del vault: ' c && echo "$c" > ~/.config/kitsalud/vault-pass; echo)
cd kitsalud/platform/ansible
ansible-vault view inventarios/kit/group_vars/all/vault.yml >/dev/null && echo "vault abierto"
```

Después se crea el archivo que usa el salto por kit01 para llegar a las máquinas virtuales, porque los dos saltos piden contraseña.

```
ansible-playbook playbooks/preparar-control.yml    # crea ~/.config/kitsalud/ssh-pass desde el vault
```

**Para un kit nuevo con secretos propios.** Se genera una contraseña del vault y se crea el archivo con las mismas claves de `vault.example.yml`. La contraseña del usuario de administración va dos veces, en claro (`vault_admin_password`) y como hash para cloud-init (`vault_admin_password_hash`).

```
(umask 077; openssl rand -base64 32 > ~/.config/kitsalud/vault-pass)
openssl passwd -6                                  # pide la contraseña del usuario kitsalud y muestra
        su hash
ansible-vault create inventarios/kit/group_vars/all/vault.yml
```

## 4. NetBird en la máquina de control

kit01 se administra por NetBird, así que la máquina de control entra en la misma red. La clave de registro sale de la consola de NetBird del grupo y no se guarda en ningún archivo.

```
curl -fsSL https://pkgs.netbird.io/install.sh | sh
 sudo netbird up --setup-key '<clave-de-registro>'   # con un espacio al inicio, para que no quede en
        el historial
netbird status | grep -E 'Management|Signal'        # Connected en los dos
```

En un kit nuevo, kit01 todavía no está en NetBird. Se agrega al instalarlo y su dirección de NetBird se escribe en `inventarios/kit/hosts.yml` (ver kit01).

## 5. Comprobar el acceso

Con el kit ya instalado, la máquina de control tiene que llegar a kit01 y, a través de él, a las dos máquinas virtuales. Ansible no acepta la clave de host de un equipo nuevo cuando entra con contraseña, así que la de kit01 se acepta una vez con `ssh`. Las de las máquinas virtuales las acepta el salto por kit01.

```
sshpass -f ~/.config/kitsalud/ssh-pass ssh -o StrictHostKeyChecking=accept-new kitsalud@100.90.225.113
        hostname   # kit01
ansible kit:vms -m ping
```

```
kit01 | SUCCESS => { "changed": false, "ping": "pong" }
clinica01 | SUCCESS => { "changed": false, "ping": "pong" }
```

```
comunidad01 | SUCCESS => { "changed": false, "ping": "pong" }
```

**Probado en un contenedor Ubuntu 24.04 recién creado,** con la red de NetBird de la máquina anfitriona y siguiendo solo esta parte. Los tres equipos respondieron `pong` y los 21 playbooks pasaron `ansible-playbook --syntax-check`.

# Equipos de red

Esta parte conecta los equipos y deja configurados el switch (sw01) y el punto de acceso (ap01). Los dos se configuran desde un portátil conectado directamente a cada uno, así que pueden quedar listos antes de instalar kit01. Su comprobación en remoto se hace en la comprobación final, cuando kit01 ya enruta la red Interna.

## Cableado

| Desde | Hasta | Qué lleva |
|---|---|---|
| **kit01** `enp170s0` | Red del sitio (uplink con Internet) | WAN. Por aquí entra también NetBird |
| **kit01** `enp171s0` | sw01 `ether1` | Trunk con las VLAN 10 (Interna), 20 (Servidores) y 40 (Comunidad), solo etiquetadas |
| **sw01** `ether2` | Puerto LAN del inyector PoE del AP | Wi-Fi de la Interna (VLAN 10) y de la Comunidad (VLAN 40) |
| **Inyector PoE, puerto PoE** | ap01 | Alimentación y datos del AP |
| **sw01** `ether4` **a** `ether8` | Estaciones de la clínica | Interna (VLAN 10) sin etiqueta |
| **Disco USB de copias** | Un puerto USB 3.0 de kit01 | Repositorio de restic |

`ether3` queda deshabilitado y del `ether9` en adelante quedan fuera del bridge.

## Switch sw01 (MikroTik CCR2004)

La configuración completa está en `network/switch/sw01.rsc`, exportada con `/export terse`. Se aplica sobre un switch sin configuración, por la consola serial, para no perder el acceso mientras cambian las VLAN.

| Qué deja | Valor |
|---|---|
| **Bridge** | `bridge-kit` con filtrado de VLAN y DHCP snooping, con `ether1-kit01` como único puerto de confianza |
| **Puertos** | `ether1-kit01` trunk, `ether2-ap01` con 10 y 40 etiquetadas y sin etiqueta en la 10 (la gestión del AP responde sin etiqueta), `ether4` a `ether8` de acceso en la VLAN 10 |
| **Gestión** | `vlan10-Interna` con `10.20.10.2/24` y `fd5a:fc7e:d716:10::2/64`, gateway `10.20.10.1` y `fe80::1` |

| Qué deja | Valor |
|---|---|
| **Servicios** | Solo SSH y Winbox, desde `10.20.10.1` y las direcciones de administración `10.20.10.10` a `10.20.10.29`. Telnet, FTP, web y API apagados |
| **Protección** | Sin reenvío IPv4 ni IPv6 y descarte de los anuncios de router (IPv6 a `ff02::1`) que entren por los puertos de acceso |
| **Hora y logs** | Zona `America/Bogota`, NTP desde `10.20.20.10` y syslog por UDP hacia `10.20.20.10` |

### Pasos

1. Conectar el cable de consola al puerto serial RJ45 del switch y abrir la terminal a 115200 8N1.

   ```
   sudo picocom -b 115200 /dev/ttyUSB0
   ```

2. Entrar como `admin` y dejar el switch sin configuración. Se reinicia solo.

   ```
   /system reset-configuration no-defaults=yes skip-backup=yes
   ```

3. Subir `sw01.rsc` al switch. Con el switch sin IP, se conecta el portátil a `ether3` y se abre Winbox por la MAC (pestaña **Neighbors**), y el archivo se arrastra a **Files**. También sirve pegar el contenido del archivo en la consola, de a unas pocas líneas.

4. Aplicarlo desde la consola serial.

   ```
   /import file-name=sw01.rsc
   ```

5. Poner la contraseña del usuario `admin`, que está en el vault como `vault_sw01_password`. El archivo no la trae, porque `/export` omite los datos sensibles.

   ```
   /user set admin password="<contraseña del vault>"
   ```

6. Conectar los cables de la tabla de cableado.

### Cómo comprobarlo en la consola

```
/interface bridge vlan print       # VLAN 10, 20 y 40 con los puertos de la tabla
/ip address print                  # 10.20.10.2/24 en vlan10-Interna
/ip service print                  # solo ssh y winbox habilitados
```

# Punto de acceso ap01 (TP-Link TL-WA801ND v3)

El AP no tiene línea de comandos, así que se configura en su interfaz web con los valores de esta tabla. Después se revisa en remoto con `network/ap/revisar-ap.sh`, que lee las mismas páginas y no cambia nada.

| Menú | Parámetro | Valor |
|---|---|---|
| **Wireless → Wireless Settings** | Modo de operación | Multi-SSID con **Enable VLAN** |
| **Wireless → Wireless Settings** | SSID1 | `SaludMovil-Clinica`, VLAN 10 |
| **Wireless → Wireless Settings** | SSID2 | `SaludMovil-Comunidad`, VLAN 40 |
| **Wireless → Wireless Settings** | SSID3 y SSID4 | Desactivados |
| **Wireless → Wireless Settings** | Región, modo y canal | Colombia, 11bgn mixed, 20 MHz, canal automático |
| **Wireless → Wireless Security** | SSID1 | WPA2-PSK con AES y la clave del Wi-Fi de la clínica |
| **Wireless → Wireless Security** | SSID2 | Sin seguridad (la protege el portal cautivo de kit01) |
| **Wireless → Wireless Advanced** | AP Isolation y Short GI | Activados |
| **DHCP → DHCP Settings** | Servidor DHCP | Desactivado (las direcciones las entrega kit01) |
| **WPS y System Tools → SNMP** | WPS y SNMP | Desactivados |
| **Network → LAN** | IP | Fija, `10.20.10.3/24`, gateway `10.20.10.1` |
| **Network → LAN** | Allow remote access | Desactivado |
| **System Tools → Password** | Contraseña de administración | La del vault, `vault_ap01_password` |

### Pasos

1. Conectar el AP con su inyector PoE solo al portátil, poner en el portátil la dirección fija `192.168.0.10/24` y abrir `http://192.168.0.254`. Si el AP ya tenía otra configuración, se restablece manteniendo el botón **Reset** unos 5 segundos con el equipo encendido.
2. Desactivar primero el servidor DHCP, WPS y SNMP.
3. Configurar el modo Multi-SSID con VLAN, los SSID, su seguridad, la región y AP Isolation.
4. Al final, cambiar la IP a `10.20.10.3`. El AP se reinicia y deja de responder en el portátil.
5. Guardar una copia de la configuración desde **System Tools → Backup & Restore**, fuera del repositorio, porque contiene la clave del Wi-Fi.
6. Conectar el puerto LAN del inyector a `ether2` de sw01.

**Cómo comprobarlo.** Con kit01 desplegado, desde `network/` en la máquina de control, `./ap/revisar-ap.sh` termina con `17 bien y 0 con falla`.

# kit01

kit01 es el mini PC Beelink EQi12 que hace de router, firewall e hipervisor. Esta parte lo instala desde cero y despliega con Ansible todo lo que corre en él antes de crear las máquinas virtuales, es decir, la red, el firewall, el DNS, la hora, el DHCP, el portal cautivo, el receptor de logs, la CA del kit y la UPS lógica.

## Lo que se necesita

- La máquina de control lista (Preparación).
- Monitor, teclado y la memoria USB con Ubuntu Server 24.04.5 LTS, solo para la instalación.
- `enp170s0` conectado a la red del sitio y `enp171s0` a `ether1` de sw01.
- Los datos de la red del sitio, que son la IP fija de kit01, el gateway y los DNS, o saber que entrega DHCP.

## 1. Instalar Ubuntu Server

En el instalador de Ubuntu Server 24.04.5 se eligen estos valores. Lo que no aparece en la tabla se deja como viene.

| Pantalla | Valor |
|---|---|
| **Red** | `enp170s0` con la IP del sitio (en el laboratorio, `192.168.160.69/24`, gateway `192.168.160.1`) y `enp171s0` sin configurar |
| **Almacenamiento** | Disco completo con **LVM**. El instalador crea `/boot/efi` de 1 GB, `/boot` de 2 GB y el grupo `ubuntu-vg` con `/` de 100 GiB, y deja el resto del grupo libre. Ese espacio libre es el que después usan los discos de las máquinas virtuales, así que `/` no se agranda |
| **Perfil** | Nombre del servidor `kit01` y usuario `kitsalud` con la contraseña del vault (`vault_admin_password`) |
| **SSH** | **Install OpenSSH server**, con contraseña |
| **Paquetes extra** | Ninguno |

Al terminar, desde la consola de kit01, se actualiza el sistema, se desactiva cloud-init para que no cambie el nombre ni la red en los arranques siguientes y se reinicia.

```
sudo apt-get update && sudo apt-get -y full-upgrade
sudo touch /etc/cloud/cloud-init.disabled
sudo reboot
```

**Cómo comprobarlo**

```
lsb_release -d                    # Ubuntu 24.04.5 LTS
ip -br link                       # enp170s0 y enp171s0, con esos nombres
sudo vgs ubuntu-vg                # VFree de unos 370 GiB
lscpu | grep -i virtualization    # VT-x
```

## 2. NetBird

Desde la consola de kit01 se instala NetBird, se registra con la clave de la consola del grupo y se retiene para que una actualización no lo reinicie a mitad de una sesión.

```
curl -fsSL https://pkgs.netbird.io/install.sh | sh
 sudo netbird up --setup-key '<clave-de-registro>'   # con un espacio al inicio
sudo apt-mark hold netbird
netbird status | grep -E 'Management|Signal|NetBird IP'
```

La dirección que muestra `NetBird IP` se escribe en `ansible_host` de kit01 en `platform/ansible/inventarios/kit/hosts.yml`. Desde aquí todo se hace en remoto desde la máquina de control, y el monitor y el teclado ya no hacen falta.

## 3. Datos del sitio en el inventario

Lo único que cambia de un sitio a otro es la WAN de kit01. Está en `platform/ansible/inventarios/kit/host_vars/kit01.yml`.

```
wan:
  ipv4: 192.168.160.69/24          # o "dhcp4: true" si el sitio entrega DHCP
  gw4: 192.168.160.1
  dns: [192.168.215.20, 192.168.215.30]
```

Las direcciones internas del kit (VLAN, prefijos IPv4 e IPv6, IP de cada servicio) están definidas una sola vez en `inventarios/kit/group_vars/all/red.yml` y no cambian con el sitio.

Después se acepta la clave de host de kit01 y se comprueba el acceso, como en la preparación.

```
cd kitsalud/platform/ansible
sshpass -f ~/.config/kitsalud/ssh-pass ssh -o StrictHostKeyChecking=accept-new kitsalud@<ip-netbird-
        de-kit01> hostname
ansible kit01 -m ping
```

## 4. Playbooks del host

Todos se ejecutan desde `kitsalud/platform/ansible`, en este orden y uno por vez. Cada uno termina con un `PLAY RECAP` que no debe tener `failed` ni `unreachable`. Si uno falla, se corrige la causa y se vuelve

a ejecutar el mismo, porque todos son idempotentes (una segunda ejecución termina en `changed=0`).

| # | Comando | Qué deja | Por qué en este lugar |
|---|---|---|---|
| **1** | `ansible-playbook playbooks/comun.yml -l kit01` | Usuario `kitsalud` en los grupos `libvirt` y `kvm`, SSH sin root y zona horaria `America/Bogota` | Es la base de todo lo demás |
| **2** | `ansible-playbook playbooks/kit01.yml` | Nombre `kit01.salud.movil`, QEMU/KVM y libvirt, la red `default` de libvirt apagada y NetBird retenido | La red `default` de libvirt tiene que estar apagada antes del firewall, porque trae su propio NAT |
| **3** | `ansible-playbook playbooks/red-kit01.yml` (ver abajo) | Firewall con nftables, VLAN, bridges de las VMs y de la Comunidad, direcciones IPv4 e IPv6 y reenvío | Los servicios que siguen escuchan en las direcciones que crea este paso |
| **4** | `ansible-playbook playbooks/dns.yml` | BIND9 con la zona `salud.movil` y sus inversas, recursivo para el kit, y la revisión de Internet que sin conexión lleva la Comunidad al portal | Las VMs resuelven nombres con él desde el primer arranque |
| **5** | `ansible-playbook playbooks/ntp.yml` | Chrony como servidor de hora del kit, que sigue sirviendo hora sin Internet | Las VMs, sw01 y el dominio necesitan la misma hora |
| **6** | `ansible-playbook playbooks/dhcp.yml` | Kea DHCPv4, Kea DHCPv6 sin estado y radvd en la Interna y en la Comunidad | Los clientes reciben dirección, DNS y hora |
| **7** | `ansible-playbook playbooks/portal.yml` | nginx con el portal cautivo y el servicio que autoriza a cada cliente de la Comunidad | La redirección y el set de clientes ya los puso el firewall |
| **8** | `ansible-playbook playbooks/registros.yml` | Receptor central de rsyslog, con una carpeta por equipo | Recibe los logs de las VMs desde su primer arranque |
| **9** | `ansible-playbook playbooks/ca.yml` | Raíz de la CA del kit y el certificado de `monitoreo` | Firma los intermedios de las VMs y los certificados de sus servicios |
| **10** | `ansible-playbook playbooks/nut.yml` | UPS lógica y el apagado ordenado `kit-apagado` | Apaga las VMs en orden cuando existan |

### Si se reinstala un kit01 que ya tenía copias (antes del paso 9)

`ca.yml` crea una raíz nueva si no encuentra la anterior, y con una raíz nueva dejan de valer los certificados que ya tienen los equipos de la brigada. Por eso, en un kit01 reinstalado, la raíz se recupera del disco de copias antes del paso 9. restic pide la clave del repositorio, que está en el vault como `vault_restic_password`.

```
sudo apt-get install -y restic
sudo mkdir -p /mnt/backup && sudo mount UUID=<backup_disco_uuid> /mnt/backup
sudo restic -r /mnt/backup/restic restore latest --tag config --include /etc/kitsalud/ca --target /
```

```
sudo umount /mnt/backup
sudo ls -l /etc/kitsalud/ca/raiz.key                # -rw------- root root
```

Después `ca.yml` encuentra la raíz y solo revisa los certificados. En un kit nuevo este paso no hace falta.

### El firewall y la red (paso 3)

Este paso es el único que puede cortar el acceso a kit01, así que se hace con cuidado. El playbook acepta siempre NetBird y SSH, compara lo nuevo con lo activo y solo recarga si cambia. Cuando recarga, programa la vuelta a la configuración anterior a los pocos minutos y solo la cancela si vuelve a entrar y NetBird sigue conectado.

En el primer despliegue todavía no existen las VMs ni Prometheus, y el playbook normalmente los usa para comprobar el firewall nuevo. Por eso la primera vez se ejecuta sin esas comprobaciones. Las siguientes veces se ejecuta sin los `-e`.

```
ansible-playbook playbooks/red-kit01.yml --check --diff \
  -e kit01_firewall_sondas=false -e '{"kit01_firewall_comprobaciones": []}'   # qué cambiaría
ansible-playbook playbooks/red-kit01.yml \
  -e kit01_firewall_sondas=false -e '{"kit01_firewall_comprobaciones": []}'   # aplicar
```

Mientras se aplica conviene tener abierta una segunda sesión SSH a kit01. Si el acceso se corta, se espera a que actúe la restauración y se vuelve a entrar.

#### Cómo comprobarlo

```
ansible kit01 -b -m shell -a 'ip -br addr; nft list tables; sysctl net.ipv4.ip_forward
        net.ipv6.conf.all.forwarding'
```

`lan0.10` con `10.20.10.1/24`, `br-com` con `10.20.40.1/24` y `br-srv` con `10.20.20.1/24`, cada uno con su dirección `fd5a:fc7e:d716:<vlan>::1/64` y `fe80::1`. `br-srv` tiene además las direcciones de los servicios de kit01, `10.20.20.10` y `fd5a:fc7e:d716:20::10`. Deben aparecer las tablas `inet filtro`, `ip nat_kit` y `bridge filtro_vms`, y el reenvío en `1` en las dos familias.

## Lo que queda instalado en kit01

| Servicio | Paquete y versión | Archivos de configuración | Unidad de systemd |
|---|---|---|---|
| **Base** | `openssh-server` del instalador | `/etc/ssh/sshd_config.d/10-comun.conf`, `/etc/hosts` | `ssh` |

| Servicio | Paquete y versión | Archivos de configuración | Unidad de systemd |
|---|---|---|---|
| **Virtualización** | `qemu-kvm` (QEMU 8.2.2), `libvirt-daemon-system` (10.0.0), `virtinst` (4.1.0) | Definiciones de las VMs en libvirt | `libvirtd`, `libvirt-guests` |
| **Acceso remoto** | `netbird` 0.80.0, retenido | La de NetBird, sin cambios | `netbird` |
| **Firewall** | `nftables` del sistema | `/etc/nftables.conf`, `/etc/nftables/quitar.nft`, `/etc/systemd/system/nftables.service.d/kit01.conf` | `nftables` |
| **Red** | `netplan.io` del sistema | `/etc/netplan/50-cloud-init.yaml`, `/etc/sysctl.d/60-kit01-router.conf` | `systemd-networkd` |
| **DNS** | `bind9` 9.18.39, retenido | `/etc/bind/named.conf.options`, `/etc/bind/named.conf.local`, `/etc/bind/zonas/`, zona RPZ activa en `/var/lib/bind/sin-internet.rpz` y `/usr/local/sbin/kit-internet` | `named`, `kit-internet.timer` (sin Internet, manda la Comunidad al portal) |
| **Hora** | `chrony` 4.5, retenido | `/etc/chrony/chrony.conf` | `chrony` |
| **DHCP e IPv6** | `kea-dhcp4-server` y `kea-dhcp6-server` 2.4.1, `radvd` 2.19, retenidos | `/etc/kea/kea-dhcp4.conf`, `/etc/kea/kea-dhcp6.conf`, `/etc/radvd.conf` | `kea-dhcp4-server`, `kea-dhcp6-server`, `radvd` |
| **Portal cautivo** | `nginx` 1.24.0, retenido | `/etc/nginx/conf.d/kitsalud.conf`, `/etc/nginx/sites-available/portal`, `/usr/local/lib/kitsalud-portal/portal.py`, `/usr/share/kitsalud-portal/` | `nginx`, `kitsalud-portal` |
| **Logs** | `rsyslog` 8.2312.0, retenido | `/etc/rsyslog.d/30-kitsalud-central.conf`, `/etc/logrotate.d/kitsalud-registros`, logs en `/var/log/kitsalud/` | `rsyslog` |
| **CA del kit** | `openssl` del sistema | `/etc/kitsalud/ca/` (la llave de la raíz solo aquí, con permisos `600`), `/etc/kitsalud/tls/` | Ninguna |

| Servicio | Paquete y versión | Archivos de configuración | Unidad de systemd |
|---|---|---|---|
| **UPS lógica** | `nut-server` y `nut-client` 2.8.1, retenidos | `/etc/nut/`, `/etc/kitsalud/nut/`, `/usr/local/sbin/kit-ups`, `/usr/local/sbin/kit-apagado` | `nut-server`, `nut-monitor`, `kit-ups-inicial` |

Los paquetes con versión fija quedan retenidos con `apt-mark hold`, así que una actualización general no los cambia. `apt-mark showhold` en kit01 los lista.

**Probado en el kit.** Los diez playbooks de esta parte, ejecutados con `--check --diff` contra kit01, terminaron en `changed=0`. El kit real no se reinstaló, porque eso cortaría el acceso remoto. La primera ejecución de `red-kit01.yml` sin las comprobaciones es la misma que se usó en el laboratorio virtual, donde la red y el firewall se desplegaron desde un Ubuntu recién instalado.

# Máquinas virtuales

Esta parte crea las dos máquinas virtuales en kit01 y despliega en ellas las aplicaciones, la publicación de contenido, las copias y el monitoreo. Todo se hace desde la máquina de control, porque las VMs solo aceptan SSH desde kit01 y Ansible salta por él.

## Lo que se necesita

- kit01 desplegado (kit01), con su salida a Internet funcionando. Las VMs descargan paquetes, imágenes de contenedores y el contenido de la biblioteca y los videos por HTTP y HTTPS.
- El disco USB de copias conectado a kit01 (paso 6).

## Las máquinas virtuales

| VM | vCPU | RAM | Discos | MAC | Direcciones |
|---|---|---|---|---|---|
| **`clinica01`** | 4 | 7 GB | 40 GB de sistema y 120 GB de datos en `/srv` | `52:54:00:20:00:11` | `10.20.20.11/24` y `fd5a:fc7e:d716:20::11/64` |
| **`comunidad01`** | 2 | 2,5 GB | 20 GB de sistema y 120 GB de datos en `/srv` | `52:54:00:20:00:12` | `10.20.20.12/24` y `fd5a:fc7e:d716:20::12/64` |
| **`prueba01`** | 1 | 512 MB | 10 GB | `52:54:00:40:00:50` | Por DHCP en la Comunidad. Hace de cliente para las pruebas y queda apagada |

Están definidas en `platform/ansible/inventarios/kit/group_vars/all/maquinas.yml`. Los discos son qcow2 independientes, con aprovisionamiento delgado, en el volumen lógico `ubuntu-vg/vms` montado en `/var/lib/libvirt/images`. Las dos VMs del kit se conectan al bridge `br-srv` y arrancan solas con kit01.

## Playbooks

Se ejecutan desde `kitsalud/platform/ansible`, en este orden. Igual que en kit01, cada uno debe terminar sin `failed` ni `unreachable`, y si uno falla se corrige y se repite el mismo.

| # | Comando | Qué deja | Por qué en este lugar |
|---|---|---|---|
| **1** | `ansible-playbook playbooks/vms.yml` | Volumen lógico de 320 GiB, imagen de Ubuntu 24.04 verificada con su `SHA256SUMS` y las tres VMs creadas con cloud-init. Después aplica a clinica01 y comunidad01 la base común (usuario `kitsalud`, SSH solo desde kit01 con ufw, hora desde kit01 y envío de logs a kit01) | Necesita los bridges, el DNS, la hora y el receptor de logs de kit01 |
| **2** | `ansible-playbook playbooks/samba.yml` | Controlador de dominio `ad.salud.movil` en clinica01, con sus grupos, usuarios y política de contraseñas, y los recursos `archivos` y `contenido` | La consulta de prerregistros, Grafana y la publicación usan el dominio o sus recursos |
| **3** | `ansible-playbook playbooks/clinica01.yml` | Docker, DHIS2 con PostgreSQL y la base de prerregistros, el intermedio de la CA de clinica01, el certificado LDAPS del dominio, la consulta de prerregistros y Caddy en HTTPS | La consulta entra con LDAPS al dominio y la base de prerregistros la usa comunidad01 |
| **4** | `ansible-playbook playbooks/comunidad01.yml` | Docker, la biblioteca (Kiwix), los videos (Jellyfin, con su contenido), el intermedio de la CA de comunidad01, el formulario de prerregistro y Caddy | El formulario escribe en la base de clinica01 |
| **5** | `ansible-playbook playbooks/publicacion.yml` | En clinica01, los archivos de la biblioteca en `contenido/zim`, el usuario y la llave de la publicación y su timer cada 10 minutos. En comunidad01, el destino que solo acepta esa llave | Copia el recurso `contenido` de clinica01 a comunidad01 y respeta los videos que puso el paso 4 |
| **6** | `ansible-playbook playbooks/backups.yml` | restic, la clave del repositorio y el timer semanal en kit01, y en cada VM el usuario `backup` que solo permite extraer | Necesita las bases, el dominio y los recursos que dejan los pasos 2 a 5 |
| **7** | `ansible-playbook playbooks/monitoreo.yml` | node_exporter en los tres equipos, y en kit01 Prometheus con las sondas, blackbox, nginx y Grafana con entrada por el dominio | Prueba todos los servicios, así que va al final |
| **8** | `ansible-playbook playbooks/red-kit01.yml --check --diff` | Nada. Comprueba que el firewall sigue igual, ahora con las sondas y los flujos entre las VMs | Desde aquí se ejecuta siempre sin los `-e` del primer despliegue |

### Disco de copias (antes del paso 6)

Si el disco es nuevo, se formatea en kit01 con una partición `kitbackup` en ext4 y su UUID se escribe en `backup_disco_uuid` de `inventarios/kit/group_vars/all/backups-equipos.yml`. Antes de borrar se confirma con `lsblk` que el dispositivo es el disco USB, por su tamaño, su serie y `TRAN usb`.

```
lsblk -o NAME,SIZE,SERIAL,TRAN,MODEL                # identificar el disco, por ejemplo /dev/sdX
sudo sgdisk -Z /dev/sdX && sudo sgdisk -n1:0:0 -t1:8300 -c1:kitbackup /dev/sdX
```

```
sudo mkfs.ext4 -m 0 -L kitbackup -E lazy_itable_init=0 /dev/sdX1
sudo blkid -s UUID -o value /dev/sdX1                  # va en backup_disco_uuid
```

El playbook no deja el disco montado. La copia lo monta solo mientras dura y, si el disco no tiene repositorio, lo crea con la clave del vault.

## Lo que queda instalado en las VMs

### En las dos

| Servicio | Paquete y versión | Archivos de configuración | Unidad de systemd |
|---|---|---|---|
| **Base** | Ubuntu 24.04.5 desde la imagen cloud `release-20260926` | `/etc/ssh/sshd_config.d/10-comun.conf`, reglas de ufw | `ssh`, `ufw` |
| **Hora** | `chrony` 4.5, retenido | `/etc/chrony/chrony.conf` con `ntp.salud.movil` como única fuente | `chrony` |
| **Envío de logs** | `rsyslog` del sistema | `/etc/rsyslog.d/90-reenvio.conf`, con cola en disco | `rsyslog` |
| **Contenedores** | `docker.io` 29.1.3 y `docker-compose-v2` 2.40.3, retenidos | `/etc/docker/daemon.json` | `docker` |
| **Monitoreo** | `prometheus-node-exporter` 1.7.0, retenido | `/etc/default/prometheus-node-exporter` | `prometheus-node-exporter` |
| **Copias** | Ninguno | `/etc/ssh/sshd_config.d/20-backup.conf`, `/etc/sudoers.d/backup`, `/usr/local/sbin/backup-extraer`, `/etc/kitsalud/backup/rsyncd.conf` | Ninguna (lo invoca kit01) |

### clinica01

| Servicio | Paquete o imagen | Archivos de configuración | Cómo corre |
|---|---|---|---|
| **Dominio y archivos** | `samba-ad-dc` 4.19.5 y `krb5-user` 1.20.1, retenidos | `/etc/samba/smb.conf`, `/etc/krb5.conf`, recursos en `/srv/samba/` | `samba-ad-dc` |
| **Historia clínica** | `dhis2/core:2.42.6.0` y `postgis/postgis:16-3.5` | `/srv/dhis2/compose.yml`, `.env`, `pg_hba.conf`, metadatos en `kit.json` | Compose `dhis2` |
| **Consulta de prerregistros** | `kitsalud/consultas:1`, construida en la VM | `/srv/consultas/compose.yml` y `.env` | Compose `consultas` |

| Servicio | Paquete o imagen | Archivos de configuración | Cómo corre |
|---|---|---|---|
| **HTTPS** | `caddy:2.11.7` | `/srv/caddy/compose.yml`, `/srv/caddy/conf/Caddyfile`, intermedio en `/srv/caddy/pki/` | Compose `caddy` |
| **Publicación** | `rsync` del sistema | `/usr/local/sbin/publicar-contenido`, llave en `/var/lib/publicacion/` | `publicacion.timer` |

### comunidad01

| Servicio | Imagen | Archivos de configuración | Cómo corre |
|---|---|---|---|
| **Biblioteca** | `ghcr.io/kiwix/kiwix-serve:3.8.2` | `/srv/kiwix/compose.yml`, archivos en `/srv/medios/zim/` | Compose `kiwix` y `kiwix-recarga.path`, que la reinicia cuando cambian los archivos |
| **Videos** | `jellyfin/jellyfin:12.3.20261023-013714` | `/srv/jellyfin/compose.yml`, videos en `/srv/medios/videos/` | Compose `jellyfin` |
| **Formulario de prerregistro** | `kitsalud/formularios:1`, construida en la VM | `/srv/formularios/compose.yml` y `.env` | Compose `formularios` |
| **HTTPS y HTTP** | `caddy:2.11.7` | `/srv/caddy/compose.yml`, `/srv/caddy/conf/Caddyfile` | Compose `caddy` |
| **Destino de la publicación** | `rsync` del sistema | `/etc/ssh/sshd_config.d/20-publicacion.conf`, `/etc/kitsalud/publicacion/rsyncd.conf` | Lo invoca clinica01 por SSH |

### En kit01, de los pasos 6 y 7

| Servicio | Paquete y versión | Archivos de configuración | Unidad de systemd |
|---|---|---|---|
| **Copias** | `restic` 0.16.4, retenido | `/usr/local/sbin/backup-kit`, `/etc/kitsalud/backup/` (llave y clave del repositorio) | `backup.timer`, el domingo a las 4:00 |
| **Métricas y sondas** | `prometheus` 2.45.3, `prometheus-blackbox-exporter` 0.24.0 y `prometheus-node-exporter` 1.7.0, retenidos | `/etc/prometheus/prometheus.yml`, `/etc/prometheus/reglas/alertas.yml`, `/etc/prometheus/blackbox.yml` | `prometheus`, `prometheus-blackbox-exporter`, `prometheus-node-exporter` |

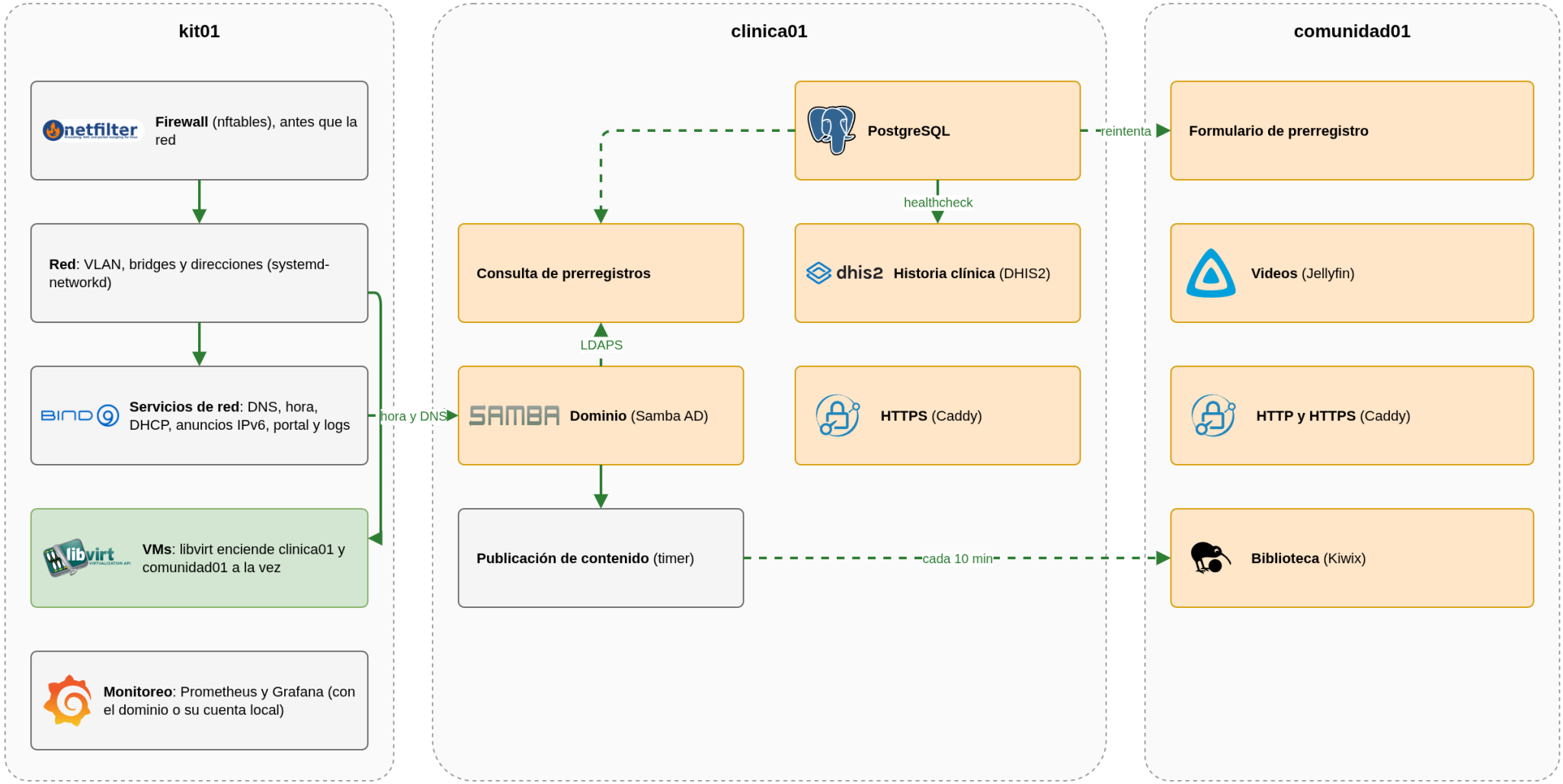
| Servicio | Paquete y versión | Archivos de configuración | Unidad de systemd |
|---|---|---|---|
| **Paneles** | `grafana` 13.2.4 desde `apt.grafana.com`, retenido | `/etc/grafana/`, paneles en `/var/lib/grafana/dashboards/`, `/etc/nginx/sites-available/monitoreo` | `grafana-server`, `nginx` |

Las imágenes de contenedores van con versión fija en cada `compose.yml`, así que volver a desplegar baja siempre la misma.

**Probado en el kit.** `vms.yml`, `samba.yml`, `publicacion.yml` y `backups.yml`, ejecutados con `--check --diff`, terminaron en `changed=0`. comunidad01 se reconstruyó desde cero con los pasos 1, 4, 5, 6 y 7, con el resultado en Reconstruir una máquina virtual.

# Orden de inicio

Esta parte explica qué arranca primero cuando se enciende el kit y de qué depende cada servicio. Al encender no hay que hacer nada a mano. El orden lo resuelven systemd en cada equipo, el arranque automático de libvirt para las VMs y los reintentos de las propias aplicaciones.


kit01
Firewall (nftables), antes que la red
Red: VLAN, bridges y direcciones (systemd-networkd)
Servicios de red: DNS, hora, DHCP, anuncios IPv6, portal y logs
VMs: libvirt enciende clinica01 y comunidad01 a la vez
Monitoreo: Prometheus y Grafana (con el dominio o su cuenta local)
hora y DNS
clinica01
PostgreSQL
reintenta
Formulario de prerregistro
healthcheck
Consulta de prerregistros
Historia clínica (DHIS2)
Videos (Jellyfin)
LDAPS
Dominio (Samba AD)
HTTPS (Caddy)
HTTP y HTTPS (Caddy)
Publicación de contenido (timer)
cada 10 min
Biblioteca (Kiwix)
comunidad01
Lo ordena systemd o Compose. La flecha va de lo que arranca primero a lo que lo necesita
La aplicación arranca igual y reintenta hasta que responde lo que necesita

## Secuencia al encender

| Fase | Equipo | Qué arranca, en orden | Tiempo medido |
|---|---|---|---|
| **1** | sw01 y ap01 | El switch carga su configuración y levanta las VLAN. El AP levanta sus dos Wi-Fi | Hasta que se encienden los indicadores de sus puertos |
| **2** | kit01, sistema | Se monta el volumen de las VMs (`/var/lib/libvirt/images`, con `nofail`). nftables carga el firewall antes de que exista la red (`Before=network-pre.target`), así nunca hay un momento sin filtro. Después systemd-networkd crea la WAN, las VLAN, los bridges y sus direcciones | 2 min 15 s en su último arranque, casi todo esperando la red |
| **3** | kit01, servicios de red | BIND9, Chrony, Kea DHCPv4 y DHCPv6 (después de la red y de la hora), radvd, rsyslog, nginx, el portal cautivo (después de nftables, porque escribe en su set) y NetBird | Segundos después de la red |
| **4** | kit01, UPS y monitoreo | `kit-ups-inicial` deja la UPS lógica en `OL` antes de que arranque su driver, y después `nut-server` y `nut-monitor`. Prometheus, blackbox y Grafana empiezan a probar los servicios | Segundos |
| **5** | kit01, VMs | libvirtd enciende clinica01 y comunidad01 a la vez, porque las dos tienen arranque automático. prueba01 queda apagada | Las VMs arrancan su sistema en unos 7 s |
| **6** | clinica01 | ufw, la red, Chrony y rsyslog. Después Samba AD y Docker (los dos esperan `network-online.target`). En Docker, PostgreSQL primero y DHIS2 cuando la base pasa su healthcheck. La consulta de prerregistros y Caddy arrancan a la vez y reintentan hasta que responden el dominio y la base | DHIS2 tarda unos 25 s con datos |
| **7** | comunidad01 | ufw, la red, Chrony y rsyslog. Después Docker con Kiwix, Jellyfin, el formulario y Caddy. El formulario reintenta hasta que responde PostgreSQL de clinica01 | Segundos |
| **8** | Todo el kit | Prometheus ve todas las sondas en verde | 42 s después de encender las VMs |

Las dos VMs arrancan juntas porque libvirt no tiene un retardo entre ellas, y no hace falta. Ninguna aplicación de comunidad01 deja de arrancar si clinica01 todavía no responde, solo reintenta. Lo mismo pasa con Grafana en kit01, que deja entrar con su cuenta local mientras el dominio no responde.

## Dependencias de cada servicio

| Servicio | Necesita | Si falta |
|---|---|---|
| **Firewall (nftables, kit01)** | Nada | Es lo primero que se carga. Con un error en el archivo, kit01 arranca sin las tablas del kit y NetBird sigue entrando |
| **DNS (BIND9, kit01)** | Las direcciones `10.20.20.10` y `fd5a:fc7e:d716:20::10` de `br-srv` | Responde en cuanto aparecen las direcciones. Los nombres de `ad.salud.movil` los delega a Samba en clinica01 |
| **Hora (Chrony, kit01)** | La red. Internet es opcional | Sin Internet sigue dando la hora local del kit a todos los equipos |
| **DHCP y anuncios IPv6 (Kea y radvd, kit01)** | `lan0.10` y `br-com` | Los clientes no reciben dirección hasta que arrancan |
| **Portal cautivo (kit01)** | nftables, por su set `portal_ok`, y nginx | La Comunidad no puede aceptar las condiciones y no pasa del portal |
| **Dominio (Samba AD, clinica01)** | La hora de kit01 (Kerberos falla con más de 5 minutos de diferencia) y su propio DNS | No entran la consulta de prerregistros, los recursos SMB ni los usuarios del dominio en Grafana |
| **PostgreSQL (clinica01)** | El disco de datos en `/srv` | DHIS2 espera su healthcheck. El formulario y la consulta reintentan |
| **DHIS2 (clinica01)** | PostgreSQL sano | No arranca hasta que la base está lista |
| **Consulta de prerregistros (clinica01)** | PostgreSQL y el dominio por LDAPS | Responde con error hasta que los dos están |
| **Formulario de prerregistro (comunidad01)** | PostgreSQL de clinica01 | Avisa al usuario y se recupera solo cuando vuelve la base |
| **Biblioteca y videos (comunidad01)** | El disco de datos en `/srv` | Funcionan aunque clinica01 esté apagada |
| **Publicación de contenido (clinica01)** | Samba (el recurso `contenido`) y el SSH de comunidad01 | La pasada falla y la siguiente, 10 minutos después, lo intenta de nuevo |
| **Logs centrales (rsyslog, kit01)** | Las direcciones de `br-srv` | Las VMs guardan los mensajes en una cola en disco y los entregan cuando vuelve |
| **Copias (restic, kit01)** | Las VMs encendidas y el disco USB | La copia de esa semana queda marcada como fallida en el monitoreo |
| **Grafana (kit01)** | nginx y, para los usuarios del dominio, Samba | Queda la cuenta local `admin` |

## Cómo comprobar el orden

```
# En kit01
systemd-analyze critical-chain multi-user.target    # qué esperó a qué en el último arranque
systemctl list-dependencies --after kitsalud-portal # el portal después de nftables
virsh -c qemu:///system list --autostart            # clinica01 y comunidad01
# En una VM (por ejemplo, desde kit01, ssh kitsalud@10.20.20.11)
systemd-analyze critical-chain docker.service       # Docker después de network-online.target
sudo docker compose -f /srv/dhis2/compose.yml ps    # db (healthy) antes que core
```

**Medido en el kit.** clinica01 llegó a `multi-user.target` en 7,6 s y comunidad01 en 6,6 s, en los dos casos con Docker esperando `network-online.target`. En un simulacro de corte de energía, todo volvió a verde 42 s después de encender las VMs.

# Comprobación final

Con todos los playbooks aplicados, estas comprobaciones confirman que el kit quedó completo. Todas se hacen en remoto desde la máquina de control, en `kitsalud/platform/ansible`, y cada una dice qué resultado esperar.

## 1. Ansible no tiene nada pendiente

Una segunda ejecución de cualquier playbook termina con `changed=0`, porque todos son idempotentes. Los de kit01 se pueden revisar con `--check --diff`, que no cambia nada.

```
for p in comun kit01 red-kit01 dns ntp dhcp portal registros ca nut vms samba publicacion backups; do
  ansible-playbook playbooks/$p.yml --check --diff -l '!pruebas' | grep -A5 'PLAY RECAP'
done
```

Los roles de las aplicaciones y de Grafana (`clinica01.yml`, `comunidad01.yml` y `monitoreo.yml`) consultan las API de DHIS2, Jellyfin y Grafana para decidir qué cambiar, y en `--check` esas consultas no se hacen, así que se detienen con `object of type 'dict' has no attribute`. Esos tres se comprueban con una ejecución normal, que también tiene que terminar en `changed=0`.

**Resultado en el kit.** Los 14 playbooks de la lista terminaron en `changed=0` en los tres equipos, es decir, el kit está exactamente como lo dejan los repositorios.

## 2. Todos los servicios en verde

Prometheus prueba cada servicio por IPv4 y por IPv6. Ninguna sonda debe estar en rojo y ningún equipo caído.

```
ansible kit01 -m shell -a "curl -s -G --data-urlencode 'query=probe_success == 0 or up == 0'
        http://127.0.0.1:9090/api/v1/query"
```

Responde `"result":[]`. En el navegador, `https://monitoreo.salud.movil` muestra el panel **Estado del kit** con **Problemas ahora** en 0. En el kit hay 44 objetivos, 39 de ellos sondas.

## 3. Los flujos del firewall

`verificar-flujos.yml` prueba cada flujo permitido y bloqueado desde su origen real (kit01, las dos VMs, `prueba01` como cliente de la Comunidad, antes y después de aceptar el portal, y sw01 como equipo de la Interna), en IPv4 e IPv6. Enciende `prueba01` y la apaga al final.

```
ansible-playbook playbooks/verificar-flujos.yml
```

Termina con `69 pruebas, todas como se esperaba`, y los contadores de los bloqueos coinciden con las líneas del log.

## 4. Switch y punto de acceso

```
# sw01: la configuración aplicada es la del repositorio (salvo la fecha del encabezado)
ssh -J kitsalud@<ip-netbird-de-kit01> admin@10.20.10.2 '/export terse' | tr -d '\r' > /tmp/sw01.rsc
diff <(grep -v '^#' ../../network/switch/sw01.rsc) <(grep -v '^#' /tmp/sw01.rsc)
# ap01: lee su interfaz web desde kit01 sin cambiar nada
(cd ../../network && ./ap/revisar-ap.sh)
```

En un switch recién configurado, `diff` no muestra diferencias. En el kit aparece además `/tool sniffer set memory-limit=200KiB`, que quedó de una prueba con el sniffer y no afecta el tráfico. `revisar-ap.sh` termina con `17 bien y 0 con falla`.

## 5. La primera copia

Con el disco USB conectado a kit01 se lanza la primera copia a mano, sin esperar al domingo.

```
ansible kit01 -b -m shell -a 'systemctl start backup.service; systemctl show -p Result --value
        backup.service'
```

Responde `success`. `journalctl -u backup -o cat | tail` muestra los cuatro snapshots (`config`, `recursos`, `bd` y `dominio`) y `restic check` sin errores.

## 6. Los servicios por su nombre

Cada nombre del kit tiene que responder en las dos familias. kit01 resuelve con los DNS del sitio, que no conocen `salud.movil`, así que la prueba pregunta a BIND9, sale con las direcciones de kit01 en la Interna y valida HTTPS con la raíz de la CA del kit.

```
ssh kitsalud@<ip-netbird-de-kit01> 'bash -s' <<'FIN'
for n in pacientes consultas registro monitoreo biblioteca videos ca; do
  case $n in biblioteca|videos|ca) u=http p=80;; *) u=https p=443;; esac
  for f in 4 6; do
    if [ $f = 4 ]; then o=10.20.10.1; ip=$(dig +short -b $o @10.20.20.10 $n.salud.movil A)
    else o=fd5a:fc7e:d716:10::1; ip="[$(dig +short -b $o @fd5a:fc7e:d716:20::10 $n.salud.movil
        AAAA)]"; fi
    echo "$n v$f $(curl -s -o /dev/null -w %{http_code} --interface $o --cacert /etc/prometheus/ca-
        kit.crt \
      --resolve $n.salud.movil:$p:$ip $u://$n.salud.movil/)"
  done
```

```
done
FIN
```

Cada línea termina en `200` o en `302`, que es la redirección a la página de inicio de sesión del servicio. En el kit respondieron así las 14.

## Errores comunes del despliegue

| Síntoma | Causa | Solución |
|---|---|---|
| **`The vault password file ... was not found`** | Falta `~/.config/kitsalud/vault-pass` | Crearlo como en la preparación |
| **`Decryption failed`** | La contraseña del vault no es la del grupo | Pedirla de nuevo por el canal privado del grupo |
| **`Using a SSH password instead of a key is not possible because Host Key checking is enabled`** | La clave de host de kit01 no está en `known_hosts` | Entrar una vez con `ssh -o StrictHostKeyChecking=accept-new` |
| **`REMOTE HOST IDENTIFICATION HAS CHANGED` hacia una VM** | La VM se volvió a crear y tiene otra clave de host | `ssh-keygen -R <ip>`. `vms.yml` lo hace solo cuando crea la VM |
| **`could not match supplied host pattern` o `the role ... was not found`** | Ansible no se ejecutó desde `platform/ansible` y no leyó `ansible.cfg` | Entrar a `kitsalud/platform/ansible` o exportar `ANSIBLE_CONFIG` con la ruta absoluta de ese archivo |
| **`red-kit01.yml` falla al consultar Prometheus o al comprobar los flujos entre las VMs** | Es el primer despliegue y todavía no existen | Ejecutarlo con `-e kit01_firewall_sondas=false -e '{"kit01_firewall_comprobaciones": []}'` |
| **Una VM no descarga paquetes ni imágenes** | kit01 todavía no tiene la salida de las VMs a Internet o la WAN del sitio está mal | Revisar `wan` en `host_vars/kit01.yml` y volver a ejecutar `red-kit01.yml` |
| **`comunidad01.yml` falla en "Originales para preparar" con `HTTP Error 429: Too many requests`** | Wikimedia limita la descarga después de varios videos seguidos y pide esperar (`Retry-After: 600`) | Esperar 10 minutos y volver a ejecutar el mismo playbook. Los videos que ya bajaron no se descargan otra vez |

| Síntoma | Causa | Solución |
| --- | --- | --- |
| **nginx no responde en un sitio después de un playbook cortado** | El corte se llevó el handler que recarga nginx | `sudo nginx -t && sudo systemctl reload nginx` en kit01. Los playbooks largos, como `monitoreo.yml` (unos 10 minutos), se ejecutan sin un tiempo límite corto |
| **DHIS2 rechaza una contraseña con `Password must not have any generic word`** | Su política rechaza palabras como `password`, `admin` o `dhis` | Cambiar esa clave en el vault por una que la política acepte |
| **`kinit` o el recurso SMB fallan con la clave correcta** | El usuario quedó bloqueado tras 5 intentos fallidos | `sudo samba-tool user unlock <usuario>` en clinica01 o esperar 15 minutos |
| **Una pasada de comprobación da `changed` sin motivo** | Se editaron archivos de un rol mientras corría, o corrían dos playbooks a la vez sobre la misma VM | Ejecutar un playbook por vez y sin editar sus archivos mientras corre |

# Reconstruir una máquina virtual

Esta parte vuelve a crear una VM desde cero cuando su disco se dañó, cuando se borró por error o cuando conviene empezar limpia. La VM nueva queda igual que la anterior, con el mismo nombre, la misma MAC y las mismas direcciones, así que el firewall, el DNS, el monitoreo y los demás equipos no cambian. Se hace en la base, con Internet, porque se descargan paquetes, imágenes de contenedores y el contenido de la biblioteca y los videos.

| VM | Qué se pierde al reconstruirla | De dónde vuelve |
|---|---|---|
| **comunidad01** | Nada propio. Su configuración sale de Ansible, la biblioteca llega con la publicación desde clinica01 y los videos los descarga su rol | Ansible y la publicación |
| **clinica01** | Los pacientes de DHIS2, los prerregistros, los archivos de los recursos SMB y lo que se haya cambiado en el dominio fuera de Ansible | La última copia de restic (sección clinica01) |
| **prueba01** | Nada. Es un cliente de pruebas | Ansible |

## Antes de empezar

1. **Copia manual**, si la VM todavía arranca. Con el disco USB conectado a kit01.

   ```
   ansible kit01 -b -m shell -a 'systemctl start backup.service; systemctl show -p Result --value
           backup.service'   # success
   ```

2. **Quitar la VM anterior.** Hay dos formas, y en las dos el nombre queda libre para crearla de nuevo. Los comandos van en kit01 (`ssh kitsalud@<ip-netbird-de-kit01>`), con `<vm>` igual a `comunidad01`, `clinica01` o `prueba01`.

   **Con vuelta atrás** (recomendada si la VM todavía funciona). Se apaga, se guarda su definición y se apartan sus discos con otro nombre. Si la reconstrucción falla, la VM anterior vuelve en un minuto.

   ```
   sudo virsh shutdown <vm>; until [ "$(sudo virsh domstate <vm>)" = "shut off" ]; do sleep 2; done
   sudo virsh dumpxml <vm> | sudo tee /root/<vm>.xml >/dev/null
   sudo virsh undefine <vm>
   cd /var/lib/libvirt/images
   for d in <vm>-sistema <vm>-datos; do [ -e $d.qcow2 ] && sudo mv $d.qcow2 $d.qcow2.anterior; done
   ```

   **Sin vuelta atrás**, cuando el disco está dañado o la VM ya no sirve.

   ```
   sudo virsh destroy <vm>; sudo virsh undefine <vm> --remove-all-storage
   ```

## comunidad01

Desde `kitsalud/platform/ansible` en la máquina de control, en este orden.

| # | Comando | Qué hace |
|---|---|---|
| **1** | `ansible-playbook playbooks/vms.yml` | Crea la VM que falta con su MAC, sus discos y su red, borra de `known_hosts` la clave de host anterior y le aplica la base común. Las otras VMs no se tocan |
| **2** | `ansible-playbook playbooks/comunidad01.yml` | Docker, la biblioteca, los videos con su contenido, el intermedio de la CA de comunidad01, el formulario de prerregistro y Caddy |
| **3** | `ansible-playbook playbooks/publicacion.yml` | Vuelve a crear el destino de la publicación y actualiza en clinica01 la clave de host de comunidad01 |
| **4** | `ansible-playbook playbooks/backups.yml` | Vuelve a crear el usuario `backup` en la VM y actualiza en kit01 su clave de host |
| **5** | `ansible-playbook playbooks/monitoreo.yml -l kit01,comunidad01` | node_exporter en la VM nueva, con su regla de ufw |

La biblioteca queda vacía hasta la primera publicación, que el timer hace en 10 minutos como máximo. Para no esperar se lanza a mano.

```
ansible clinica01 -b -m shell -a 'systemctl start publicacion.service; systemctl show -p Result --
        value publicacion.service'   # success
```

Si el paso 2 se detiene en "Originales para preparar" con `HTTP Error 429`, Wikimedia está limitando las descargas de los videos. Se esperan 10 minutos y se vuelve a ejecutar el mismo paso.

**Probado en el kit.** comunidad01 se reconstruyó desde cero siguiendo solo esta parte, con la VM anterior apartada para poder volver atrás.

| Paso | Duración | Resultado |
|---|---|---|
| **Copia manual y VM anterior apartada** | 47 s | `success` en 17 s. Discos apartados con `.anterior` |
| **1. `vms.yml`** | 1 min 14 s | VM nueva en 74 s. clinica01 en `changed=0` |
| **2. `comunidad01.yml`** | 28 min | La primera pasada se detuvo con `HTTP Error 429` en tres videos de Wikimedia. Después de esperar los 10 minutos que pide (`Retry-After: 600`), terminó en 4 min 44 s |
| **3, 4 y 5. `publicacion.yml`, `backups.yml` y `monitoreo.yml`** | 2 min | Sin fallas. En clinica01 y en kit01 solo cambió la clave de host de comunidad01 |

| Paso | Duración | Resultado |
|---|---|---|
| **Publicación manual** | Menos de 1 min | Llegaron los 3 archivos de la biblioteca y Kiwix se reinició solo |

Después de la reconstrucción, comunidad01 tenía lo mismo que antes, es decir, los 3 archivos de la biblioteca, los 8 videos y audios y sus 4 contenedores sanos. Las 39 sondas estaban en verde, `verificar-flujos.yml` dio `69 pruebas, todas como se esperaba` y la copia manual terminó en `success` con `restic check` sin errores. Desde `prueba01`, como un celular de la Comunidad, la biblioteca, los videos, el registro y la CA respondieron en IPv4 e IPv6, el certificado del registro salió del intermedio nuevo de comunidad01 y dos prerregistros de prueba se guardaron en la base de clinica01. Una segunda ejecución de `comunidad01.yml` terminó en `changed=0`, y `red-kit01.yml --check` también. Los servicios de comunidad01 estuvieron fuera 31 minutos, 20 de ellos esperando a Wikimedia, y la clínica siguió funcionando todo el tiempo.

## clinica01

clinica01 tiene los datos del kit, así que después de recrearla se restauran desde la última copia. Los pasos de Ansible son los mismos del despliegue, en este orden.

| # | Comando | Qué hace |
|---|---|---|
| **1** | `ansible-playbook playbooks/vms.yml` | Crea la VM y le aplica la base común |
| **2** | `ansible-playbook playbooks/samba.yml` | Aprovisiona otra vez el dominio con sus grupos, sus usuarios y sus uid y gid fijos, y crea los recursos `archivos` y `contenido` |
| **3** | `ansible-playbook playbooks/clinica01.yml` | DHIS2 con una base nueva, la base de prerregistros, el intermedio de la CA, el certificado LDAPS, la consulta y Caddy |
| **4** | Restaurar los datos (abajo) | Pacientes, prerregistros y archivos |
| **5** | `ansible-playbook playbooks/publicacion.yml` | Origen de la publicación, los archivos de la biblioteca y el timer |
| **6** | `ansible-playbook playbooks/backups.yml` | Usuario `backup` en la VM y su clave de host en kit01 |
| **7** | `ansible-playbook playbooks/monitoreo.yml -l kit01,clinica01` | node_exporter y el acceso de Grafana al dominio |

El dominio se aprovisiona de nuevo con Ansible, porque solo contiene lo que crea Ansible. Como los uid y gid de los usuarios son fijos, los archivos restaurados conservan sus dueños. Se pierden las contraseñas que los usuarios hayan cambiado y las cuentas creadas a mano, que se vuelven a crear.

**Restaurar los datos.** En kit01, como root, con el disco USB conectado.

```
sudo -i
mount -o noatime,nodev,nosuid,noexec UUID=<backup_disco_uuid> /mnt/backup
export RESTIC_REPOSITORY=/mnt/backup/restic RESTIC_PASSWORD_FILE=/etc/kitsalud/backup/restic.clave
        RESTIC_CACHE_DIR=/var/cache/restic
S=/var/lib/kitsalud/backup
restic snapshots                                                    # elegir la copia, por defecto
        latest
restic dump latest --tag bd $S/clinica01/postgres/dhis2.dump > /tmp/dhis2.dump
restic dump latest --tag bd $S/clinica01/postgres/prerregistro.dump > /tmp/prerregistro.dump
restic restore latest --tag recursos --include $S/clinica01/archivos --target /tmp/r
umount /mnt/backup
scp /tmp/dhis2.dump /tmp/prerregistro.dump kitsalud@10.20.20.11:/tmp/
scp -r /tmp/r$S/clinica01/archivos kitsalud@10.20.20.11:/tmp/archivos
```

En clinica01 (`ssh kitsalud@10.20.20.11` desde kit01), con DHIS2 detenido para que no escriba durante la carga.

```
cd /srv/dhis2 && sudo docker compose stop core
sudo docker exec -i dhis2-db-1 pg_restore -U dhis -d dhis2 --clean --if-exists < /tmp/dhis2.dump
sudo docker exec -i dhis2-db-1 pg_restore -U dhis -d prerregistro --clean --if-exists <
        /tmp/prerregistro.dump
sudo docker compose start core
sudo cp -a /tmp/archivos/. /srv/samba/archivos/
sudo rm -rf /tmp/dhis2.dump /tmp/prerregistro.dump /tmp/archivos
```

Los archivos heredan la ACL por defecto del recurso, así que quedan con los mismos permisos que antes. El recurso `contenido` no hace falta restaurarlo si la publicación ya tenía todo en comunidad01. Si se perdió, se restaura igual que `archivos`. Las tablas analíticas de DHIS2 se regeneran solas a las 2:00.

Estos comandos son los de la prueba de restauración en el kit, donde la base `dhis2` completa volvió con sus 333 tablas y sus 45 893 filas en 7,7 s, una tabla de `prerregistro` volvió con el mismo md5 y un archivo de `archivos` volvió con su dueño del dominio y su ACL. La reconstrucción completa de clinica01 no se hizo en el kit real, porque detiene la historia clínica durante una o dos horas.

## prueba01

```
ansible-playbook playbooks/vms.yml
ansible-playbook playbooks/prueba01.yml -e prueba01_estado=apagada
```

prueba01 no lleva la base común ni ufw, porque hace de celular de la Comunidad.

## Si la reconstrucción falla

Con la VM anterior apartada, se borra la nueva y se vuelve a poner la anterior. En kit01, con `<vm>` igual al nombre de la VM.

```
sudo virsh destroy <vm>; sudo virsh undefine <vm> --remove-all-storage
cd /var/lib/libvirt/images
```

```
for d in <vm>-sistema <vm>-datos; do [ -e $d.qcow2.anterior ] && sudo mv $d.qcow2.anterior $d.qcow2;
        done
sudo virsh define /root/<vm>.xml && sudo virsh autostart <vm> && sudo virsh start <vm>
```

Desde la máquina de control se borra la clave de host de la VM nueva (`ssh-keygen -R <ip>`) y se ejecutan `publicacion.yml` y `backups.yml`, que vuelven a registrar la clave de la VM anterior.

**Cuando la VM nueva funciona,** se borran la definición y los discos apartados, que ocupan espacio en el volumen de las VMs.

```
sudo rm /root/<vm>.xml /var/lib/libvirt/images/<vm>-*.qcow2.anterior
```

## Cómo comprobar que quedó bien

Son las mismas comprobaciones del despliegue, todas en remoto.

```
ansible kit01 -m shell -a "curl -s -G --data-urlencode 'query=probe_success == 0 or up == 0'
        http://127.0.0.1:9090/api/v1/query"   # "result":[]
ansible-playbook playbooks/verificar-flujos.yml
        # 69 pruebas, todas como se esperaba
ansible kit01 -b -m shell -a 'systemctl start backup.service; systemctl show -p Result --value
        backup.service'                    # success
ansible kit01 -b -m shell -a 'ls /var/log/kitsalud/<vm>/ && tail -1 /var/log/kitsalud/<vm>/syslog.log'
        # logs recientes de la VM
```